Facade

Facade · One placeOrder call over inventory+payment+shipping · Build with Waqa

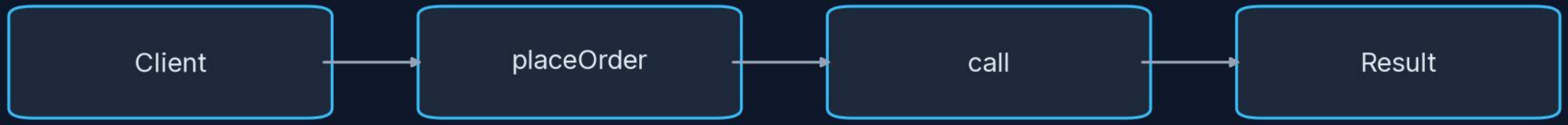
Today's Hook

- Today's force: One placeOrder call over inventory+payment+shipping
- We earn Facade by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: Inventory, Payments, Shipping, OrderFacade

The Problem

- Scenario: One placeOrder call over inventory+payment+shipping
- Constraints: solve «One placeOrder call over inventory+payment+shipping» with Facade — keep the demo runnable and the failure path observable.
- Naive pain: client calls three services itself
- Success looks like: OK:SKU

Structure



Facade: One placeOrder call over inventory+payment+shippin

Real-World Scenario

- Running example: One placeOrder call over inventory+payment+shipping
- Key types: Inventory, Payments, Shipping, OrderFacade
- Happy-path stdout cue: OK:SKU
- Failure-path stdout cue: FAIL:payment

Naive Code

```java
package com.buildwithwaqar.patterns.gof.structural.facade.problem;

import java.util.*;

public final class NaiveCheckout {
  public static String place(String sku, int cents) {
    // client calls three services itself
    boolean inv = true; boolean pay = cents > 0; boolean ship = true;
    if (!inv || !pay || !ship) return "FAIL";
    return "OK:"+sku;
  }
}
```

Why Naive Hurts

- Client calls three services itself
- Without Facade, call sites absorb the wrong details.
- Failure mode to show on camera: FAIL:payment
- New requirements force shotgun edits across the naive code.

Solution

```
package com.buildwithwaqar.patterns.gof.structural.facade.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;

final class Inventory { boolean reserve(String sku){ return sku!=null && !sku.isBlank(); } void release(String sku){} }
final class Payments { boolean charge(int cents){ return cents>0; } }
final class Shipping { boolean ship(String addr){ return addr!=null; } }
final class OrderFacade {
  private final Inventory inv=new Inventory();
  private final Payments pay=new Payments();
  private final Shipping ship=new Shipping();
  String placeOrder(String sku, int cents, String addr){
    if (!inv.reserve(sku)) return "FAIL:inventory";
    if (!pay.charge(cents)) { inv.release(sku); return "FAIL:payment"; }
    if (!ship.ship(addr)) return "FAIL:shipping";
    return "OK:"+sku;
  }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.structural.facade.problem;

import java.util.*;
import java.util.concurrent.*;

public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Facade: placeOrder over inventory+payment+shipping ===");
    System.out.println("--- naive ---");
System.out.println(NaiveCheckout.place("SKU", 100));
System.out.println("--- facade ---");
var f=new OrderFacade();
System.out.println(f.placeOrder("SKU", 100, "X"));
System.out.println(f.placeOrder("SKU", 0, "X"));
  }
}
```

Walkthrough Output

```
=== Facade: placeOrder over inventory+payment+shipping ===
--- naive ---
OK:SKU
--- facade ---
OK:SKU
FAIL:payment
```

Step-by-Step Trace

- Match stdout to stages — look for: OK:SKU
- Then the injected failure: FAIL:payment
- Re-run: java -cp ... 12-facade ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
// Java 25 direction for Facade
// Prefer records + sealed where they shrink the demo.
// Problem under test: One placeOrder call over inventory+payment+shipping
```

Trade-offs

- Use Facade when «One placeOrder call over inventory+payment+shipping» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Facade in product language.

Related Patterns

- Name the pattern usually confused with Facade and separate the forces.
- Sketch the roles for «One placeOrder call over inventory+payment+shipping» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: One placeOrder call over inventory+payment+shipping
- Naive pain → Facade boundary → Demo stdout.
- Remember the failure cue: FAIL:payment
- Break a ProblemTest assertion, watch it fail, restore it.